

SQL

TRANSACTIONS + TABLE LOCKING

Ing. Ladislav Körösi, PhD.
ÚRK, FEI STU

ladislav.korosi@stuba.sk

ÚVOD

A transaction is a unit of work that is performed against a database. Transactions are units or sequences of work accomplished in a logical order, whether in a manual fashion by a user or automatically by some sort of a database program.

A transaction is the propagation of one or more changes to the database. For example, if you are creating a record or updating a record or deleting a record from the table, then you are performing transaction on the table. It is important to control transactions to ensure data integrity and to handle database errors.

Properties of Transactions

Transactions have the following four standard properties, usually referred to by the acronym ACID:

- **Atomicity:** ensures that all operations within the work unit are completed successfully; otherwise, the transaction is aborted at the point of failure, and previous operations are rolled back to their former state.
- **Consistency:** ensures that the database properly changes states upon a successfully committed transaction.
- **Isolation:** enables transactions to operate independently of and transparent to each other.
- **Durability:** ensures that the result or effect of a committed transaction persists in case of a system failure.

Transaction Control

There are following commands used to control transactions:

- **COMMIT**: to save the changes.
- **ROLLBACK**: to rollback the changes.
- **SAVEPOINT**: creates points within groups of transactions in which to ROLLBACK
- **SET TRANSACTION**: Places a name on a transaction.

Transactional control commands are only used with the DML commands INSERT, UPDATE and DELETE only. They can not be used while creating tables or dropping them because these operations are automatically committed in the database.

COMMIT

The COMMIT command is the transactional command used to save changes invoked by a transaction to the database.

The COMMIT command saves all transactions to the database since the last COMMIT or ROLLBACK command.

Syntax:

COMMIT;

COMMIT

Example:

```
DELETE FROM CUSTOMERS  
WHERE AGE = 25;
```

```
COMMIT;
```

Following is the example which would delete records from the table having age = 25 and then COMMIT the changes in the database. As a result, two rows from the table would be deleted.

(COMMIT is the default TRANSACTION)

ROLLBACK

The ROLLBACK command is the transactional command used to undo transactions that have not already been saved to the database.

The ROLLBACK command can only be used to undo transactions since the last COMMIT or ROLLBACK command was issued.

Syntax:

ROLLBACK;

ROLLBACK

Example:

```
DELETE FROM CUSTOMERS  
WHERE AGE = 25;
```

```
ROLLBACK;
```

Following is the example, which would delete records from the table having age = 25 and then ROLLBACK the changes in the database. As a result, delete operation would not impact the table.

ROLLBACK

Syntax of correct using of ROLLBACK:

```
SET autocommit=0;
```

```
START TRANSACTION;
```

```
Your Query here.
```

```
ROLLBACK;
```

SAVEPOINT

A SAVEPOINT is a point in a transaction when you can roll the transaction back to a certain point without rolling back the entire transaction.

Syntax:

```
SAVEPOINT SAVEPOINT_NAME;
```

This command serves only in the creation of a SAVEPOINT among transactional statements. The ROLLBACK command is used to undo a group of transactions

Syntax:

```
ROLLBACK TO SAVEPOINT_NAME;
```

SAVEPOINT

Following is an example where you plan to delete the three different records from the CUSTOMERS table. You want to create a SAVEPOINT before each delete, so that you can ROLLBACK to any SAVEPOINT at any time to return the appropriate data to its original state.

Example:

```
SAVEPOINT SP1;  
DELETE FROM CUSTOMERS WHERE ID=1;  
SAVEPOINT SP2;  
DELETE FROM CUSTOMERS WHERE ID=2;  
SAVEPOINT SP3;  
DELETE FROM CUSTOMERS WHERE ID=3;
```

SAVEPOINT

Now that the three deletions have taken place, say you have changed your mind and decided to ROLLBACK to the SAVEPOINT that you identified as SP2. Because SP2 was created after the first deletion, the last two deletions are undone:

Example:

ROLLBACK TO SP2;

Notice that only the first deletion took place since you rolled back to SP2

SAVEPOINT

The `RELEASE SAVEPOINT` command is used to remove a `SAVEPOINT` that you have created.

Syntax:

```
RELEASE SAVEPOINT SAVEPOINT_NAME;
```

Once a `SAVEPOINT` has been released, you can no longer use the `ROLLBACK` command to undo transactions performed since the `SAVEPOINT`.

SET TRANSACTION

The SET TRANSACTION command can be used to initiate a database transaction. This command is used to specify characteristics for the transaction that follows.

For example, you can specify a transaction to be read only, or read write.

Syntax:

```
SET TRANSACTION [ READ WRITE | READ ONLY ];
```

Table Locking

A lock is a flag associated with a table. MySQL allows a client session to explicitly acquire a table lock for preventing other sessions from accessing the same table during a specific period.

A client session can acquire or release table locks only for itself. And a client session cannot acquire or release table locks for other client sessions.

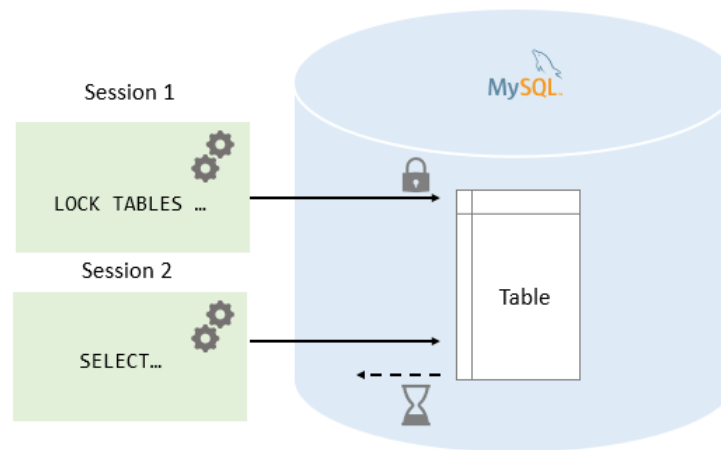


Table Locking

Syntax for lock:

```
LOCK TABLES table_name [READ | WRITE];
```

```
LOCK TABLES table_name1 [READ | WRITE],  
               table_name2 [READ | WRITE],  
               ... ;
```

Syntax for unlock:

```
UNLOCK TABLES;
```

Table Locking

A READ lock has the following features:

- A READ lock for a table can be acquired by multiple sessions at the same time. In addition, other sessions can read data from the table without acquiring the lock.
- The session that holds the READ lock can only read data from the table, but cannot write. And other sessions cannot write data to the table until the READ lock is released. The write operations from another session will be put into the waiting states until the READ lock is released.
- If the session is terminated, either normally or abnormally, MySQL will release all the locks implicitly. This feature is also relevant for the WRITE lock.

Table Locking

A WRITE lock has the following features:

- The only session that holds the lock of a table can read and write data from the table.
- Other sessions cannot read data from and write data to the table until the WRITE lock is released.

Table Locking

A WRITE lock has the following features:

- The only session that holds the lock of a table can read and write data from the table.
- Other sessions cannot read data from and write data to the table until the WRITE lock is released.

Table Locking

Syntax to show the process list in MySQL:

```
SHOW PROCESSLIST;
```

Syntax to read the connection ID:

```
SELECT CONNECTION_ID();
```

See more at:

<https://www.mysqltutorial.org/mysql-table-locking/>